Part 1: Introduction to Python

Dr. Christian Debes
christian@debes.de

Signal Processing Group
Technische Universität Darmstadt

What is Python?

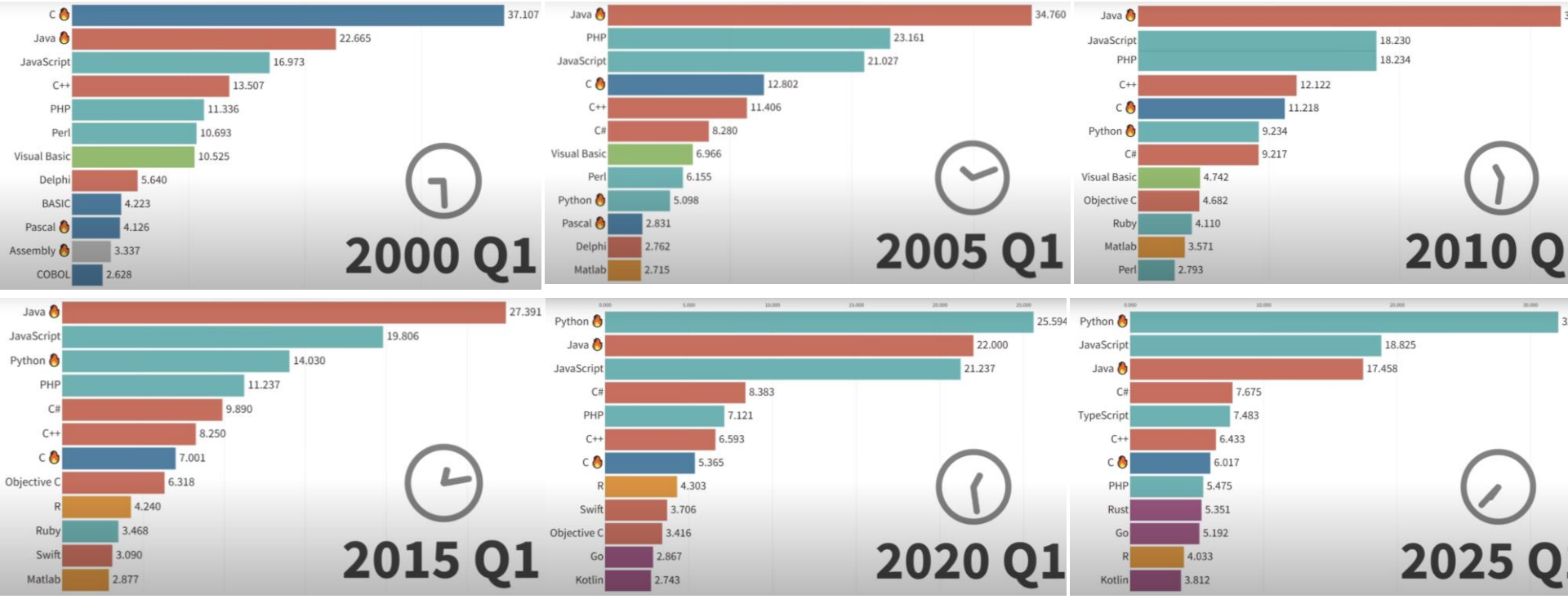
- Python is a programming language
- Invented by Guido van Rossum
- The first version was released in 1991
- Interpreted language (no need to compile before execution)
- ..which makes it portable (doesn’t depend on the OS you use)
- ..but it makes it slower than a compiled language

Why do you need to learn Python?

- Really easy to learn and intuitive
- Easy switch if you know Matlab or R
- Wide range of libraries available
- Efficient code (possibility to write complex code in just few lines)
- Number 1 language for Machine Learning, AI and Big Data
- Increasingly used in many companies (try searching for data scientist job offers)

+ Huge community, lot’s of open source projects to help you !

Python among other programming languages



Job evolvement - 2009

Job Description (2009)

We are seeking a Data Scientist with strong Python programming skills to join our research team. The ideal candidate will have extensive experience with statistical analysis and predictive modeling techniques.

Key Responsibilities:

- Develop and implement statistical models using Python
- Research and test novel algorithms for pattern recognition
- Create data visualizations to communicate research findings
- Document methodologies and publish research findings

Required Skills:

- PhD in Statistics, Mathematics, Computer Science, or related field
- 3+ years of experience with Python programming for statistical analysis
- Expert knowledge of NumPy and SciPy packages
- Proficiency with matplotlib for research visualization
- Strong background in statistical modeling and hypothesis testing

Job evolvement - 2025

Job Description (2025)

We are seeking a versatile Senior Data Scientist to drive business impact through our advanced analytics and AI initiatives. The ideal candidate will bridge the gap between data science, engineering, and business domains while leveraging modern tools and platforms.

Key Responsibilities:

- Develop and deploy end-to-end machine learning and AI solutions that deliver measurable business value
- Orchestrate data science workflows across cloud environments
- Design and implement solutions using both traditional ML and large language models

Required Skills:

- Master's degree or higher in a quantitative field (Computer Science, Data Science, Statistics, etc.)
- 3+ years of professional data science experience
- Working knowledge of Python and key data science libraries
- Experience deploying and managing AI solutions on AWS or similar cloud platforms
- Understanding of containerization and Kubernetes for scalable deployments
- Hands-on experience with LLMs, prompt engineering, and fine-tuning approaches
- Ability to translate business problems into data science solutions

Preferred Qualifications:

- Experience with MLOps practices and tools
- Familiarity with responsible AI frameworks and governance
- Knowledge of Vector databases and retrieval-augmented generation
- Experience with real-time analytics systems
- Background in one or more domain specialties (finance, healthcare, manufacturing, etc.)
- Certification in cloud platforms or AI/ML specializations

Illustration of “Why Python ?”

- C++

```cpp
#include <iostream>

int main() {
    std::cout << "Hello World!";
    return 0;
}
```

- Java

```java
class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

- Python

```python
print('Hello, world!')
```

How to use Python?

- We strongly recommend you to download the last version of python.
- In general, new versions are made to be compatible with older ones (ex: 3.1, 3.2, 3.3…).
- However some scripts in Python 2 won’t work with Python 3
- Moreover, since the 1st January 2020, Python 2 is not developed anymore

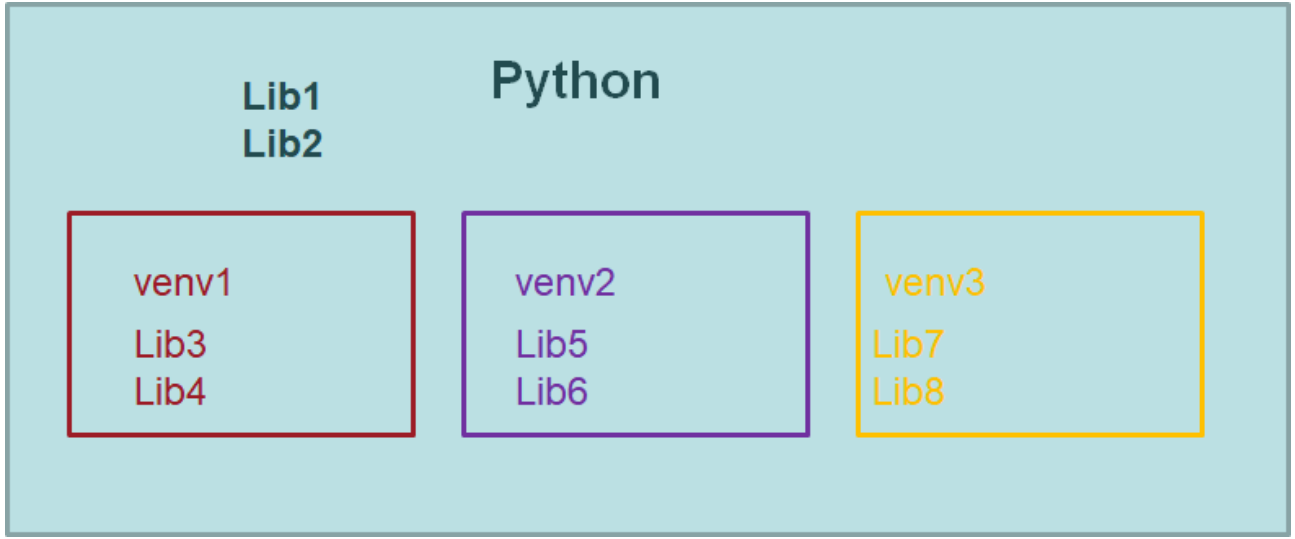
Where to write in Python?

- Some IDEs: Spyder , Pycharm, VSCode

- We will use the jupyter notebook during our tutorials.

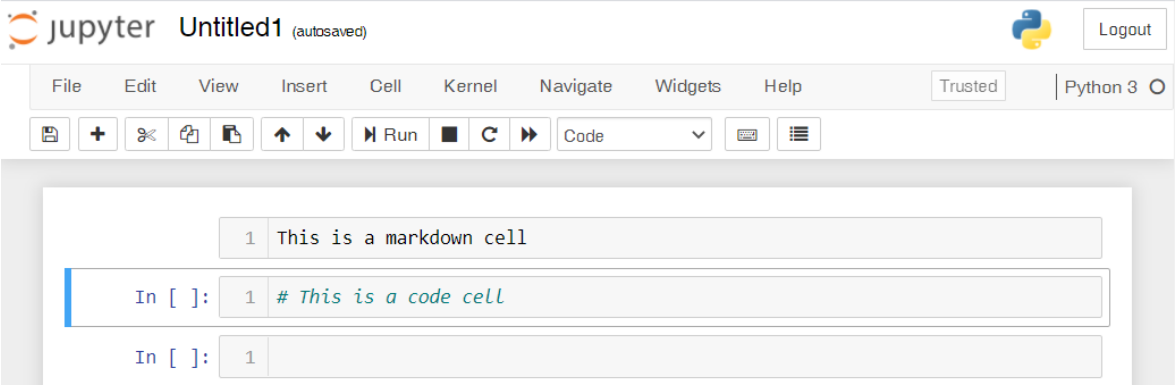
Virtual environment (Venv)

- Local environment to keep all your project separate.
- For example it avoids some project version overlapping, etc



Why Jupyter?

- It can generate a full blown report, made possible by the markdown cells
- It can support 40 different languages
- The cells make it very easy to test your code and correct it
- Very elegant to document a whole project



Why Jupyter?

- The cells can be run independently. You can run the second cell before the first one.

The code will not run in the order of the cells

but in the order of execution!

If you change a variable in the first cell that is reused in other cell, you have to re-execute those cells to update the value.

Libraries

Python itself contains a lot of functionalities. Those can be easily extended depending of the type of project you are working on. To name some:

numpy for scientific computing (efficient array and matrix computations, linear algebra functions, ...)

scipy also for scientific computing (statistical, optimization, integration, signal processing functions, …)

matplotlib and ***seaborn*** for data plotting

pandas for exploration and manipulation of data

sklearn for machine learning

Need help?

- Stack Overflow
- docs.python.org
- help() function
- … your favorite chatbot

Jupyter Notebook Tips (for Mac, replace Ctrl with Command):
(Esc)A to insert cell above
(Esc)B to insert cell below
(Esc)DD to delete selected cell
Enter to enter edit mode
Ctrl-Enter to run cell
Shift-Enter to run cell and select below
Alt-Enter to run cell and insert below

Data Types

To declare a variable :

variable= 1 ⇒ integer

variable= 1. ⇒ float

variable= “1” ⇒ string

If you want to know the type of variable: type(a)

```
In [1]:  x = 1000
         type(x)
Out[1]:  int

In [2]:  y = 1.5
         type(y)
Out[2]:  float

In [3]:  z = 1,5
         type(z)
Out[3]:  tuple

In [4]:  travel = False
         type(travel)
Out[4]:  bool

In [5]:  message0 = 'No'
         message1 = 'Yes'
         type(message1)
Out[5]:  str
```

Comparison Operators

In [1]:
```python
# Comparison of booleans
print(True == False)
```
```
False
```

In [2]:
```python
# Comparison of integers
print((-5*5)!=25)
```
```
True
```

In [3]:
```python
# Comparison of strings
print('name'=='Name')
```
```
False
```

In [4]:
```python
# Compare a boolean with an integer
print(True==1)
print(False==0)
```
```
True
True
```

In [5]:
```python
# Comparison of integers
x = (-3)**2
print(x >= 8)

x = -3**2
print(x >= 8)

# parentheses matter!
```
```
True
False
```

In [6]:
```python
# Comparison of strings
y = 'abc'
print('ab' <= y)
print('abc' <= y)
print('abcd' <= y)
```
```
True
True
False
```

In [7]:
```python
# Comparison of booleans
print(True > False)
print(True <= False)
```
```
True
False
```

Lists

- A list can contain a mix of variable types.

In [1]:
```
movie_ratings = ['Toy Story', 4.0,
                 'Jumanji', 4.0,
                 'Grumpier Old Men', 4.0,
                 'Waiting to Exhale', 5.0,]
print(movie_ratings)
```

```
['Toy Story', 4.0, 'Jumanji', 4.0, 'Grumpier Old Men', 4.0, 'Waiting to Exhale', 5.0]
```

- You can make lists within a list.
- .append() vs .extend(), subsetting, slicing, removing in tutorial.
- List comprehension:

In [2]:
```
x = [0,1,2,3,4]

# List comprehension
[i%2 for i in x]
```

Out[2]: [0, 1, 0, 1, 0]

Mentimeter

Tuples

- Immutable collection of items, separated by commas, in a single variable
- Can be assigned with or without round brackets
- The **zip() function** returns an iterator of tuples

In [1]:
```
titles = ('Toy Story',
          'Jumanji',
          'Grumpier Old Men',
          'Waiting to Exhale')
print(titles[0])
titles[0] = 'Toy Story 2'
```

```
Toy Story

---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-1-10cdda36af7a> in <module>
      4           'Waiting to Exhale')
      5 print(titles[0])
----> 6 titles[0] = 'Toy Story 2'

TypeError: 'tuple' object does not support item assignment
```

In [2]:
```
titles = ['Toy Story',
          'Jumanji',
          'Grumpier Old Men',
          'Waiting to Exhale']
ratings = [4.0, 4.0, 4.0, 4.0]
movie_data = zip(titles, ratings)
```

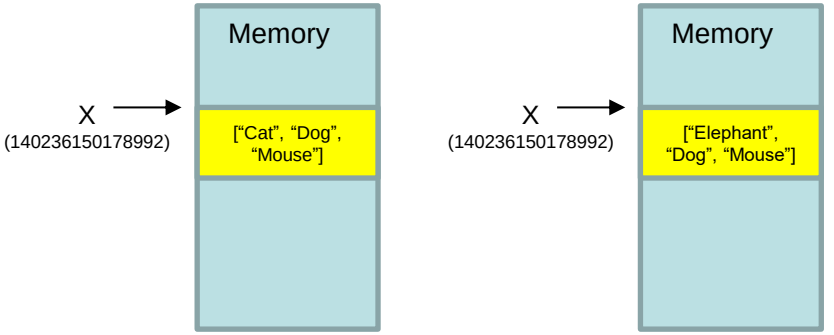
In [3]:
```
# iterate over movie_data
for idx, data in enumerate(movie_data, 1): # to start at 1 instead of 0
    mx, rx = (data)
    print('{}: {}, rating: {}'.format(idx, mx, rx))
```

```
1: Toy Story, rating: 4.0
2: Jumanji, rating: 4.0
3: Grumpier Old Men, rating: 4.0
4: Waiting to Exhale, rating: 4.0
```

Lists vs Tuples – a quick excursion

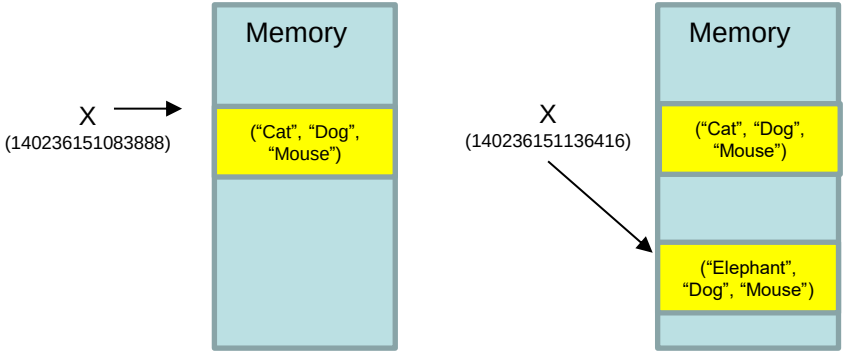
List

```
In [14]: x = ["Cat", "Dog", "Mouse"]
         id(x)
Out[14]: 140236150178992

In [15]: x[0] = ["Elephant"]
         id(x)
Out[15]: 140236150178992
```



Tuple

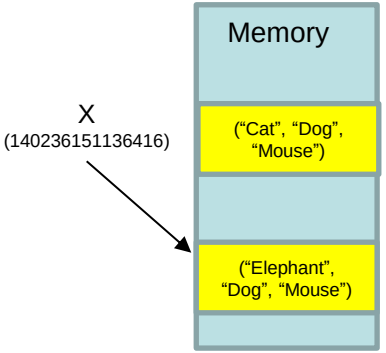
```
In [8]: x = ("Cat", "Dog", "Mouse")
        id(x)
Out[8]: 140236151083888

In [9]: x = ("Elephant", "Dog", "Mouse")
        id(x)
Out[9]: 140236151136416
```



Lists vs Tuples – a quick excursion

```
In [8]: x = ("Cat", "Dog", "Mouse")
        id(x)

Out[8]: 140236151083888

In [9]: x = ("Elephant", "Dog", "Mouse")
        id(x)

Out[9]: 140236151136416
```

x is a variable. A variable refers to a python object in memory



The code on the left side will generate a new python object to which x is referencing now

Python has garbage collection, so the old python object will be deleted

- Lists (mutable) are better when frequently modifying an object, e.g. through appending
- Tuples (immutable) can be more memory-efficient (Python can bundle variables) and can be better for debugging

Dictionary

- Pairs of keys and values

```
In [1]:
movie_ratings = {'Toy Story':4.0,
                 'Jumanji':4.0,
                 'Grumpier Old Men':4.0}

movie_ratings
Out[1]: {'Toy Story': 4.0, 'Jumanji': 4.0, 'Grumpier Old Men': 4.0}

In [2]:
movie_ratings.keys()
Out[2]: dict_keys(['Toy Story', 'Jumanji', 'Grumpier Old Men'])

In [3]:
movie_ratings.values()
Out[3]: dict_values([4.0, 4.0, 4.0])
```

- We can use the **zip()** function to create a dictionary from lists

```
In [4]:
titles = ['Toy Story', 'Jumanji', 'Grumpier Old Men']
ratings = [4.0, 4.0, 4.0, 4.0, 5.0]
movie_ratings = {key:value for (key,value) in zip(titles, ratings)}
movie_ratings
Out[4]: {'Toy Story': 4.0, 'Jumanji': 4.0, 'Grumpier Old Men': 4.0}
```

For and while loops

- Mind the colon ':', indentation, and range.

In [1]:
```
for i in range(3):
    print(i)
```

```
0
1
2
```

In [2]:
```
i = 1
while i < 3:
    print('{} squared is: {}'.format(i,i**2))
    i = i+1
```

```
1 squared is: 1
2 squared is: 4
```

- return (in a function), continue,

  and break terminate the loop.
  - continue: to stop the current iteration
  - break: to stop all iterations

In [3]:
```
def num():
    for number in range(1, 10):
        if number%2 == 0:
            return number
num()
```

```
2
```

If, elif, else

- Use if, elif, else to add condition(s) of execution.
- Condition is not enclosed by parentheses.
- Can be nested.
- Again, mind the colon ‘:’ and the indentation.

In [1]:

```
def cgpa_note(cgpa):
    if cgpa<1.60:
        print("sehr gut")
    elif 1.60<=cgpa<2.60:
        print("gut")
    elif 2.60<=cgpa<3.60:
        print("befriedigend")
    elif 3.60<=cgpa<4.10:
        print("ausreichend")
    else:
        print("nicht ausreichend")
```

In [2]:

```
cgpa_note(1.60)
```

```
gut
```

map, filter, reduce

- While they can be replaced by list comprehension or loops, `map()`, `filter()`, and `reduce()` provide an elegant and short-hand approach.

- `map()` executes a function on the given iterable items
  `map(function_to_apply, list_of_inputs)`

- `filter()` creates a list of elements for which a function returns true
  `filter(function_as_rule, list_of_inputs)`

- `reduce()` applies a rolling computation to sequential pairs of values in a list
  `reduce(function_to_apply, list_of_inputs)`

Functions and lambda expressions

- A function can be defined with no/one/multiple parameters and can return no/one/multiple values.
- Lambda expressions allow efficient coding.
  - Useful for e.g. in combination with map(), filter(), reduce()

In [1]:
```
def modulo(a,b):
    return a%b
```

In [2]:
```
modulo(3,2)
```

Out[2]: 1

In [3]:
```
modulo = lambda a,b : a%b
```

In [4]:
```
modulo(3,2)
```

Out[4]: 1

In [5]:
```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]

# using map
mod3_all = map(lambda num: num%3, numbers)
# same with list comprehension [num%3 for num in numbers]
list(mod3_all)
```

Out[5]: [1, 2, 0, 1, 2, 0, 1, 2]

In [6]:
```
# using filter
even = filter(lambda num: num%2==0, numbers)
list(even)
```

Out[6]: [2, 4, 6, 8]

In [7]:
```
# using reduce
from functools import reduce
result = reduce(lambda num1, num2: num1+num2, numbers)
result
```

Out[7]: 36

Error Handling

TECHNISCHE UNIVERSITÄT DARMSTADT

By try-except

In [1]:
```
def cube(x):
    """Returns cube of a number"""
    try:
        return x**3
    except:
        print('the argument must be a number')
```

In [2]:
```
cube("hello")
```

```
the argument must be a number
```

By raising an error (TypeError, ValueError)

In [3]:
```
from math import sqrt
def sqrt_chk(x):
    """Returns square root of a number"""
    if type(x)==str:
        raise TypeError("Won't work with a string")
    if x<0:
        raise ValueError("Let's not deal with imaginary numbers")
    return sqrt(x)
```

In [4]:
```
sqrt_chk(-4)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-4-cb10c9432ae1> in <module>
----> 1 sqrt_chk(-4)

<ipython-input-3-64fa667ba041> in sqrt_chk(x)
      5         raise TypeError("Won't work with a string")
      6     if x<0:
----> 7         raise ValueError("Let's not deal with imaginary numbers")
      8     return sqrt(x)

ValueError: Let's not deal with imaginary numbers
```

Modules and Packages

- A module is a file containing Python definitions and statements.
  - a module can be imported into other modules or into the main module
- Packages are a way of structuring Python's module namespace
  - `A.B` designates a submodule named `B` in a package named `A`
- Aliasing modules

```
import [module] as [another_name]
```

e.g.

```
import numpy as np
import matplotlib.pyplot as plt
```

For readability, mind the conventions!

NumPy

- A library that supports efficient computation of large, multidimensional arrays and matrices.

In [1]:

```python
import numpy as np
np.random.seed(123)
A = np.random.randint(-5,5, size=(2,5))
B = np.random.randint(-3,3, size=(2,5))
print(type(A))
print("A:\n {}".format(A))
print("B:\n {}".format(B))
print('Item size:', A.itemsize) # length in bytes of one element
print('Size:', A.size)
print('Size in bytes:', A.nbytes) # total bytes consumed
print('Data Type of A: ', A.dtype)
print('Dimension of A: ', A.ndim)
print('Shape of A: ', A.shape)
```

```
<class 'numpy.ndarray'>
A:
 [[-3 -3  1 -4 -2]
 [ 4  1 -4 -5 -4]]
B:
 [[-2 -3 -3 -2  0]
 [ 2  1 -3 -3  1]]
Item size: 4
Size: 10
Size in bytes: 40
Data Type of A:  int32
Dimension of A:  2
Shape of A:  (2, 5)
```

- ndarray: N-dimensional array type

https://numpy.org/

NumPy basics

- Accessing/slicing a 2D array: `twoD_arr[row][col]` or `twoD_arr[row,col]`
- To retain the 2D shape we can use double brackets “[[ ]]” for a simple case or the method `.reshape((nrow,ncol)`. Use “-1” to infer the remaining dimension.

In [2]:
```
print(A)
```
```
[[-3 -3  1 -4 -2]
 [ 4  1 -4 -5 -4]]
```

In [3]:
```
print(A[0])
print(A[0].shape)
```
```
[-3 -3  1 -4 -2]
(5,)
```

In [4]:
```
print(A[[0]])
print(A[[0]].shape)
```
```
[[-3 -3  1 -4 -2]]
(1, 5)
```

In [5]:
```
print(A[1,::2])
print(A[1,::2].shape)
```
```
[ 4 -4 -4]
(3,)
```

In [6]:
```
print(A[1,::2].reshape(1,-1))
print(A[1,::2].reshape(1,-1).shape)
```
```
[[ 4 -4 -4]]
(1, 3)
```

In [7]:
```
print(A[:,::2])
print(A[:,::2].shape)
```
```
[[-3  1 -2]
 [ 4 -4 -4]]
(2, 3)
```

In [8]:
```
print(A[:,::2].reshape(1,-1))
print(A[:,::2].reshape(1,-1).shape)
```
```
[[-3  1 -2  4 -4 -4]]
(1, 6)
```

NumPy basics

- Matrix point-wise addition, subtraction, multiplication, and division.

In [1]:

```python
import numpy as np
np.random.seed(123)
A = np.random.randint(1,3, size=(2,3))
B = np.random.randint(1,3, size=(2,3))
print(A)
print(B)
```

```
[[1 2 1]
 [1 1 1]]
[[1 2 2]
 [1 2 2]]
```

In [2]:

```python
print(np.add(A,B))
print(np.subtract(A,B))
print(np.multiply(A,B))
print(np.divide(A,B))
```

```
[[2 4 3]
 [2 3 3]]
[[ 0  0 -1]
 [ 0 -1 -1]]
[[1 4 2]
 [1 2 2]]
[[1.  1.  0.5]
 [1.  0.5 0.5]]
```

NumPy basics

- Matrix/vector multiplications
  - The inner dimensions must match for `np.matmul()` and `np.dot()`.
  - The dimensions must match for `np.inner()`.

In [4]:
```
np.random.seed(123)
a = np.random.randint(1,5, size=(1,3))
b = np.random.randint(1,5, size=(3,1))
print(a)
print(b)
```

```
[[3 2 3]]
[[3]
 [1]
 [3]]
```

In [5]:
```
print(np.matmul(a,b))
print(np.matmul(b,a))
```

```
[[20]]
[[9 6 9]
 [3 2 3]
 [9 6 9]]
```

In [6]:
```
print(np.dot(a,b))
print(np.dot(b,a))
```

```
[[20]]
[[9 6 9]
 [3 2 3]
 [9 6 9]]
```

In [7]:
```
# print(np.inner(a,b)) will give error
print(np.inner(a,b.T))
print(np.inner(a.T,b))
```

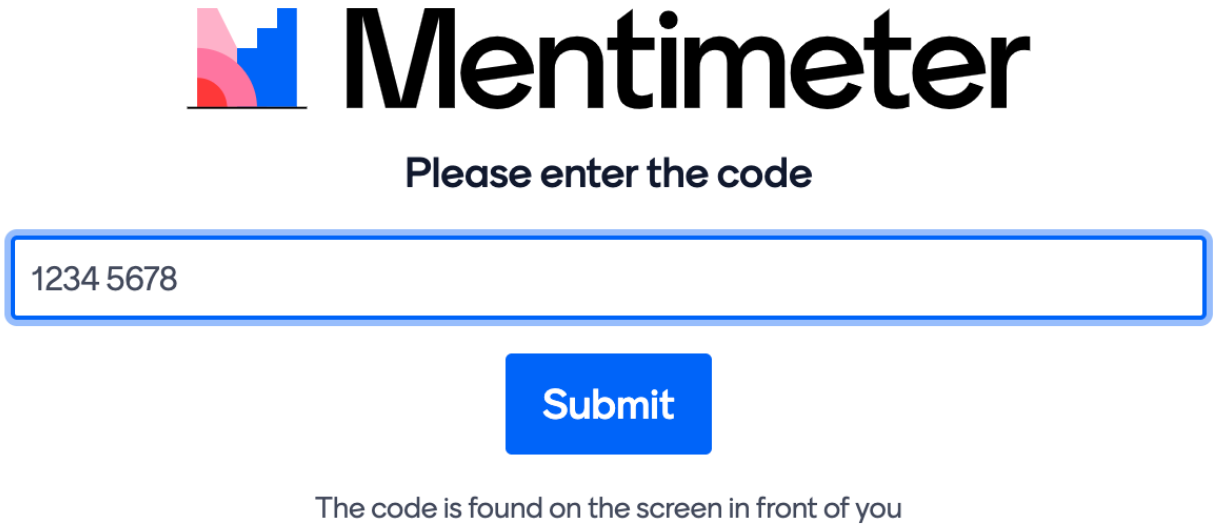
```
[[20]]
[[9 3 9]
 [6 2 6]
 [9 3 9]]
```

What should I know by now?

You should know:

- what some common libraries (numpy, pandas, matplotlib, seaborn, scipy, sklearn) are for
- what Jupyter notebook is and how to start writing with it
- some basic data types including list, tuple, and dictionary and how to access (slice/subset) and iterate through them
- how to write a function and use map, reduce, and filter with a lambda function
- how to write an error handler
- some basic Numpy functions for vector/matrix calculations

Mentimeter


Mentimeter
Please enter the code
1234 5678
Submit
The code is found on the screen in front of you